# Local Models as Coding Workers and Retrieval Engines on a Single Workstation

An Overnight Measurement on Real Tasks, with the Instrumentation Built to Repeat It

Matthias Nott

Independent Research

Lab Report

Measurement run of 14–15 October 2026

15 October 2026

## Abstract

Can open-weight language models running on one desktop workstation take over the work that cloud models do today in an agentic software workflow, where a strong model plans and smaller worker models write code? And which local models should serve a document-management system on the same machine? We answer both questions with measurements taken in a single night on an Apple M5 Max with 128 GB of unified memory. For coding, we replayed eight fixes that had already landed in a native PDF application: each worker started from the commit before the fix, received either the maintainer's own words or the orchestrator's written specification, and was graded by the hidden test that came with the real fix. With a specification, the cloud worker (Claude Sonnet 5.5) passed 6 of 7 tasks (86 %; 95 % Wilson interval (Wilson, 1927) 49–97 %), the best local model, Qwen3-Coder-30B-A3B, passed 3 of 19 (16 %; 6–38 %) and Qwen3.6-35B-A3B passed 1 of 17 (6 %; 1–27 %). The local models succeeded only on one-function fixes whose location the task named; their dominant failure was editing the visible call site instead of the code that owns the behaviour. Serving, not the models, was the first bottleneck: a prefix-cache defect made every local turn re-read its whole prompt, and fixing it cut time-to-first-token from 28–31 s to 0.6–6 s. For document retrieval, local models are sufficient and preferable: Qwen3-Embedding-0.6B reached a recall@10 of 0.950 against 0.825 for the incumbent and embedded 1.8 times faster than the most accurate alternative. We describe the method, the instrumentation that makes the comparison repeatable with one command, and a routing policy that keeps coding in the cloud and retrieval at home.

# Contents

**1 Introduction** **5**

**2 Setting** **5**
2.1 Hardware . . . . . 5
2.2 Two workloads . . . . . 6
2.3 Orchestration . . . . . 6

**3 Models** **6**

**4 Serving: the First Bottleneck** **8**

**5 Method: Replaying Fixes That Already Landed** **9**
5.1 Why replay . . . . . 9
5.2 Task construction . . . . . 9
5.3 Two kinds of prompt . . . . . 9
5.4 Isolation . . . . . 9
5.5 Grading . . . . . 10

**6 Instrumentation for Selection, Testing and Routing** **10**

**7 Results: Coding** **12**
7.1 Pass rates . . . . . 12
7.2 Per task . . . . . 12
7.3 Time and effort . . . . . 13
7.4 Why the local runs failed . . . . . 13

**8 Results: Document Retrieval** **14**

**9 Discussion** **15**

**10 Routing Policy** **15**

**11 Limitations** **16**

**12 Conclusion and Future Work** **17**

**References** **18**

## List of Tables

1 Models served during the measurement . . . . . 7
2 Serving measurements and the changes they led to . . . . . 8
3 The eight replay tasks (Project A) . . . . . 10

4 Instrumentation built during the measurement (PAI) . . . . . 11
5 Pass rates by worker and prompt (hidden tests) . . . . . 12
6 Specification prompt: passed / runs per task . . . . . 12
7 Median effort per specification run . . . . . 13
8 Failure classes of the failed local specification runs . . . . . 13
9 Embedding models for search (40 queries) . . . . . 14
10 Routing decision after the measurement . . . . . 16

# 1 Introduction

Agentic coding tools such as Claude Code (Anthropic, 2026a) let one model plan a change, delegate the editing to worker sessions, and review what comes back. In the setup studied here, the planning model (the *orchestrator*) is Claude Opus 5.5 and the workers are, by default, Claude Sonnet 5.5 (Anthropic, 2026b). Every worker turn is a cloud request against a subscription budget. A workstation with 128 GB of unified memory can hold open-weight models of the 30-billion-parameter class with room to spare, so the obvious question is whether the workers can run locally: to conserve the subscription, to keep working offline or on battery, and to keep private documents on the machine.

The question has two halves that behave differently. *Coding* needs a model that can find the right place in an unfamiliar code base, change it, build, test, and report only what it actually verified. *Document management* needs an embedding model for search and a small generative model for extraction and grounded answers; it runs in bursts, and its documents must not leave the machine. The operator's instruction for the night was explicit: understand, measure and optimise local workers on real work, not on synthetic examples; if the local models make systematic errors, fix them systematically; keep the decision reversible, including for a possible move to a 32 GB machine.

This report makes four contributions:

1. A replay benchmark built from fixes that already landed, graded by their own hidden tests, with isolation strong enough that a worker cannot look up the answer (Section 5).
2. Measurements of three local coding models against the cloud worker, with failure analysis (Section 7), and of local embedding and generation models for retrieval (Section 8).
3. The serving fixes that turned local inference from unusable to merely slow (Section 4).
4. Open-source instrumentation in PAI (Nott, 2026b) that makes the comparison repeatable for the next model with one command, and a routing policy built on it (Section 6).

# 2 Setting

## 2.1 Hardware

All measurements ran on one Apple MacBook Pro with an M5 Max (40 GPU cores) and 128 GB of unified memory. macOS limits the memory the GPU may wire by default; the operator raised it to 112 GB (`sysctl iogpu.wired_limit_mb=114688`) for the night. Two physical variables proved first-order and are reported with every result: *thermal pressure* (read with `notifyutil -g com.apple.system.thermalpressurelevel`) and *memory pressure* (swap and compression). At thermal level 2 the GPU clock fell to 569 MHz and every local measurement ran at roughly one third of its unthrottled speed; forcing the fans on returned the machine to level 0.

### 2.2 Two workloads

The two workloads are anonymised here.

*Project A* is a native PDF reader and annotation application for macOS and iPadOS, written in Swift on PDFKit, with an Xcode build of several minutes and XCTest suites. At the time of the measurement it comprised about 239,000 lines (188,000 of them Swift), counted over source, scripts and configuration with test fixtures, resources and notes excluded. The tasks of the benchmark are real faults from its 3D-annotation, ink and logging code, fixed on the days before the measurement.

*Project B* is a document-management system: a Python ingestion pipeline, a FastAPI server, PostgreSQL with pgvector (Kane, 2026) for vector search, and a Flutter client. Its model jobs are embedding every text chunk, answering questions over retrieved chunks, extracting dates and fields, and classifying documents. Project B's documents are private and must stay local in every configuration. At the time of the measurement its live index held 30,591 documents in 113,515 text chunks.

### 2.3 Orchestration

Workers are started through PAI (Nott, 2026b), an open-source layer over Claude Code that runs each worker as a separate, visible terminal session with its own git worktree, a time limit, a provider (cloud or a local OpenAI/Anthropic-compatible endpoint) and a structured final report. Sessions for the two projects exchanged requests and findings through AIBroker (Nott, 2026a), which delivers messages between Claude Code sessions. In this study the Project B session owned the model servers and the Project A session owned the coding benchmark; the orchestrator in each was Claude Opus 5.5.

## 3 Models

Table 1 lists the models that were served during the night. All local coding candidates are mixture-of-experts models (Shazeer et al., 2017) with about 3 billion parameters active per token, which is what makes a 30–80-billion-parameter model fast enough on one machine. Two of them use a hybrid architecture that interleaves linear-attention layers (Gated DeltaNet; S. Yang, Kautz, and Hatamizadeh, 2025) with ordinary attention, which matters for prefix caching (Section 4).

Local models were served with vllm-mlx (vllm-mlx contributors, 2026), an MLX-based (Hannun et al., 2023) server that speaks both the OpenAI and the Anthropic message API, so Claude Code can drive a local model directly. It offers continuous batching, chunked prefill and a prefix cache, ideas known from vLLM's PagedAttention (Kwon et al., 2023) and SGLang's RadixAttention (Zheng et al., 2024). Project B's generative model ran on Ollama (Ollama, 2026), which loads models on demand and unloads them when idle.

***Table 1:*** *Models served during the measurement*

| Model | Parameters (total / active) | Architecture | Size | Role and outcome |
|---|---|---|---|---|
| Claude Sonnet 5.5 (Anthropic, 2026b) | – | cloud | – | Reference coding worker |
| Qwen3-Coder-30B-A3B (Qwen Team, 2025a; A. Yang et al., 2025) | 30B / 3B | MoE, attention | 17 GB (4-bit) | Main local coding candidate (“local30”) |
| Qwen3.6-35B-A3B (Qwen Team, 2026) | 35B / 3B | MoE, hybrid DeltaNet | ∼20 GB (4-bit) | Second local candidate (“qwen36”) |
| Qwen3-Coder-Next (Qwen Team, 2025b) | 80B / 3B | MoE, hybrid | 45 GB (4-bit) | Early candidate; dropped: slower, no better |
| Devstral Small 2 (Mistral AI, 2025) | 24B dense | attention | ∼14 GB (4-bit) | Dropped: ∼20 s per turn |
| gpt-oss-120b (OpenAI, 2025) | 117B / 5B | MoE, attention | 59 GB (MXFP4) | Dropped: 60 s to first token, tool calls not parsed |
| multilingual-e5-base (Wang et al., 2024) | 0.28B | encoder | 0.4 GB | Incumbent embedding model (Project B) |
| bge-m3 (Chen et al., 2024) | 0.57B | encoder | 0.6 GB | Embedding candidate |
| Qwen3-Embedding-0.6B (Zhang et al., 2025) | 0.6B | decoder-based | 0.6 GB | Embedding candidate; chosen |
| Qwen3-8B (A. Yang et al., 2025) | 8B dense | attention | 4.7 GB (Q4) | Project B answers and extraction |

# 4 Serving: the First Bottleneck

Before any model could be judged, the serving stack had to be made to work under an agentic load. A coding worker in Claude Code sends a long system prompt (13–24 thousand tokens) on every turn, plus the growing conversation, and three workers ran at a time. Table 2 summarises what was measured and changed.

***Table 2:*** *Serving measurements and the changes they led to*

| Measurement | Result | Consequence |
|---|---|---|
| Qwen3-Coder-Next, 32k-token prompt, thermal level 2 vs 0 | prefill 536 vs 1,213 tok/s; decode 16 vs 55 tok/s | Thermal state reported with every number; fans forced on |
| LM Studio, three 36k-token requests at once | first tokens after 93, 168, 242 s (serial prefill) | Rejected for parallel workers |
| vllm-mlx, same three requests, continuous batching | all three first tokens after 82 s | Adopted |
| Prefix cache under real Claude Code workers | 0 hits / 41 misses; 28–31 s to first token per turn | Defect found and patched (see text) |
| After the cache patch, same load | 117 hits / 5 misses (96 %); 0.6–6.1 s to first token | Turns 5–10 times faster |
| Runaway decoding in one turn | ∼50,000 tokens at 70 tok/s | Server-side output cap of 8,192 tokens |
| Two servers each reserving 90 % of GPU memory | Metal out-of-memory, then swap thrashing | Budget of 40 % each; at most two servers |

*The prefix-cache defect.* The decisive finding of the night was that the prefix cache never hit. Claude Code inserts small *system* messages into the conversation on every turn. The server folded all system messages into the head of the prompt, so the prompt changed at token 4,400 of 13,700 on every turn and the cache could never reuse more than that. Keeping the system messages where they occur (and, for model templates that forbid a system message in mid-conversation, folding them into the next user message instead) made the whole previous prompt reusable. A second change refreshed the cache's snapshots every 512 tokens instead of every 4,096. Measured live under three concurrent workers, the hit rate went from 0 to 96 % and time to first token from 28–31 s to 0.6–6.1 s per turn.

*Robustness patches.* Three further patches were applied to the server launcher: an output cap of 8,192 tokens per request (Claude Code ignores its own output limit for model names it does not recognise and asks for 64,000, which allowed a repetition loop to run for minutes); conversion of an aborted generation into an "overloaded" error that Claude Code retries instead of hanging; and a schema-driven repair of tool-call arguments, renaming parameters

the local model guessed (for example `path` for `file_path`) to the keys the tool's schema declares. All patches are in the launcher, not in the models, so they apply to any model served this way.

*Hardware acceleration.* The M5's GPU contains matrix units that MLX uses for prefill; Apple reports up to four times faster time to first token than the previous generation and a smaller gain for decoding, which is bound by memory bandwidth (Apple Machine Learning Research, 2026). This matches what we saw: prefill was the cost that the cache fix removed, and decode speed (15–70 tokens per second depending on load and thermal state) was what remained.

# 5 Method: Replaying Fixes That Already Landed

## 5.1 Why replay

Public benchmarks such as SWE-bench (Jimenez et al., 2024) measure whether a model can resolve an issue in an open repository. They do not tell us whether a model can work in *this* code base, with its build, its conventions, its test layout and its tooling. The operator asked for measurement on the project's own recent work. The obvious source is the project's history: every fix that landed together with a test has a known correct answer and a mechanical judge.

## 5.2 Task construction

Each task is a directory holding the commit to start from (the parent of the landed fix), the landed fix itself (used only for grading and calibration), the fix's tests as *hidden* files, a deadline, and one or more prompts. Eight tasks were built from fixes of the preceding two days (Table 3). Every task was *calibrated* before use: the landed fix must pass its hidden test and the parent must fail it.

## 5.3 Two kinds of prompt

Each task carried two prompts. The *operator* prompt contains only what the maintainer had at the time: the symptom in their own words and the readings taken before the fix, never the cause. The *specification* prompt is what the orchestrator actually wrote when it delegated the fix: the diagnosis and the part of the code that owns the behaviour, but not the code of the fix. The difference between the two measures how much of the work is finding the cause and how much is making the change. A third, more precise variant naming the exact file and function was prepared for four tasks but not run (Section 11).

## 5.4 Isolation

The first runs revealed a leak: a worker in a git worktree could read other branches with `git log -all` and found the answer that another worker had committed. From then on, every run started in its own clone containing only the parent commit's history, and the hidden tests were overlaid only at grading time. Runs that never started (for example because the pane gate refused them) are recorded as *void* and excluded; they had first been counted as failures, which the grader now prevents.

***Table 3:*** *The eight replay tasks (Project A)*

| Task | Kind | Deadline | Landed fix | What the worker had to do |
|---|---|---|---|---|
| box-press | patch | 25 min | 2 files, +79 | Make a click on a 3D annotation reach the 3D viewer instead of the generic mark handling |
| view-decimals | patch | 20 min | 1 file, +6 | A saved 3D view made the file unreadable: write its numbers as plain decimals, not exponents |
| save-model-ref | patch | 40 min | 3 files, +64 | Each save copied the embedded 3D model; make saves refer to it |
| log-size | patch | 15 min | 2 files, +21 | The debug log never rotated: find why its size was read wrongly |
| ink-baked | patch | 25 min | 2 files, +28 | Ink became opaque after the eraser was used on it |
| lost-3dd | patch | 40 min | 4 files, +161 | Recover a 3D model whose reference was lost by an earlier save |
| merge-3d | merge | 30 min | 2 files, +9 | Merge two branches with a real conflict and keep both behaviours |
| diagnose-growth | report | 15 min | 3 files, +64 | Explain why each save grows the file; graded on seven named facts |

### 5.5 Grading

A *patch* task passes when the result builds and the hidden tests pass. A *merge* task passes when both branches' tests pass after the merge. A *report* task passes when the worker's report contains at least six of seven named facts and no code was touched. The grade is mechanical; no model judges another. For each run we record wall time, the number of turns, the tokens, and whether the run reached its deadline.

## 6 Instrumentation for Selection, Testing and Routing

The operator's standing rule was that systematic errors of local models must be fixed systematically, and that the comparison must be repeatable when the next model appears. Everything in Table 4 was built during the night, is part of PAI (Nott, 2026b), and was installed and in use for the later batches. The benchmark itself lives in the project's repository, because the build and test commands are project-specific; PAI only orchestrates.

Each instrument answers a failure that was observed, not anticipated. The guards, for instance, exist because local workers left scratch Swift files in the repository's top level in three tasks, deleted a README while making a two-line fix, and deleted 88 lines of an unrelated file, which broke the build. The proof check exists because a local worker reported success with `git diff HEAD` as its "proof", and the hidden test then failed.

***Table 4:*** *Instrumentation built during the measurement (PAI)*

| Component | Purpose and behaviour |
|---|---|
| `pai worker bench` | Runs a project's replay suite for any set of providers: `-providers anthropic,local30 -prompt spec -repeat 2 -max-parallel 2 -max-load 60`. Waits while the machine's load is above the limit, records the thermal level per run, and writes a table of pass rate (void runs excluded), median wall time and median turns. `-dry-run` prints the plan; `-report` only aggregates. |
| `pai worker policy` | Routing: `cloud-only`, `local-only`, `prefer-local`, `prefer-cloud` or `auto`, with per-run overrides (`-urgent`, `-local-ok`, `-cloud`) and fall-through from a failed local attempt to the cloud. |
| Provider contracts | A per-provider addendum to the worker's instructions (for example: find a file before reading it; put scratch files outside the repository), recorded by checksum with every run. |
| Proof check | A worker's "done" stands only if the proof command it names was actually run and succeeded in that session; otherwise the report is downgraded. Compound commands, directory prefixes and trailing filters are normalised. |
| Guards for local workers | Refuse, for local providers only: new files in the repository's top level, rewrites that delete more than half of a tracked file, and deletion of tracked files. |
| Timeout clamp | A shell command asking for more than two minutes is clamped to two minutes with an explanation instead of refused. |
| Visibility gate | No worker runs without a verified visible terminal pane, so every run can be watched and replayed. |
| Launcher patches | Prefix-cache fix, output cap, aborted-generation retry and tool-argument repair (Section 4); live counters of repairs. |

# 7 Results: Coding

## 7.1 Pass rates

Table 5 gives the headline numbers. Each run counts once, with its last grade; void runs are excluded. The operator-prompt rows include the early evening runs made before the serving and instruction fixes, which favours no one in particular but lowers the local numbers somewhat.

***Table 5:*** *Pass rates by worker and prompt (hidden tests)*

| | Operator prompt | | Specification prompt | | |
|---|---|---|---|---|---|
| Worker | Passed | Rate | Passed | Rate | 95 % interval |
| Claude Sonnet 5.5 (cloud) | 8 / 11 | 73 % | 6 / 7 | 86 % | 49–97 % |
| Qwen3-Coder-30B-A3B (local) | 2 / 30 | 7 % | 3 / 19 | 16 % | 6–38 % |
| Qwen3.6-35B-A3B (local) | 0 / 14 | 0 % | 1 / 17 | 6 % | 1–27 % |

The intervals are Wilson score intervals (Wilson, 1927), which stay inside 0–100 % and remain usable for small samples and rates near zero, where the usual normal approximation fails; with samples this small they are wide, but they do not overlap between the cloud worker and either local model. The difference is not noise.

## 7.2 Per task

Table 6 breaks the specification runs down by task. Local runs were repeated more often than cloud runs because their variance was the open question.

***Table 6:*** *Specification prompt: passed / runs per task*

| Task | Sonnet 5.5 | Qwen3-Coder-30B | Qwen3.6-35B |
|---|---|---|---|
| box-press | 1 / 1 | 1 / 4 | 1 / 4 |
| log-size | 1 / 1 | 2 / 3 | 0 / 3 |
| save-model-ref | 1 / 1 | 0 / 3 | 0 / 2 |
| ink-baked | 1 / 1 | 0 / 3 | 0 / 2 |
| lost-3dd | 1 / 1 | 0 / 2 | 0 / 2 |
| diagnose-growth | 1 / 1 | 0 / 2 | 0 / 2 |
| view-decimals* | 0 / 1 | 0 / 2 | 0 / 2 |
| Total | 6 / 7 | 3 / 19 | 1 / 17 |

* The specification for view-decimals named the wrong cause; the cloud worker followed it and failed as well. The row measures the specification, not the workers. It has since been corrected.

Two observations stand out. First, the local passes are all on the two smallest, most local fixes: box-press (a condition in one predicate) and log-size (one function that read a cached file size). Every task that required changing more than one file, merging, or writing a diagnosis was failed by both local models in every run. Second, the specification helped the cloud worker more than the local ones: on lost-3dd it turned a 35-minute failure into a 15-minute pass, and on diagnose-growth it raised the report from five to all seven facts in under a minute. The local pass rate rose only from 2 of 44 to 4 of 36 runs.

### 7.3 Time and effort

Table 7 shows the median wall time and the median number of turns (model calls) for the specification runs. Local medians are censored by the deadlines: most failing local runs ended because their time ran out, so the true time they would have needed is longer.

***Table 7:*** *Median effort per specification run*

| Worker | Median wall time | Median turns | Turns per minute |
|---|---|---|---|
| Claude Sonnet 5.5 | 15.3 min | 27 | 1.8 |
| Qwen3-Coder-30B-A3B | 18.8 min | 36 | 1.9 |
| Qwen3.6-35B-A3B | 22.5 min | 26 | 1.2 |

The cloud figure is inflated by the shared build queue: when several workers built Project A at once, a single build took up to 16 minutes. On the smallest task, log-size, the cloud worker finished in 2.4 minutes; the two passing local runs took 8.9 and 15.0 minutes.

### 7.4 Why the local runs failed

Every failed local specification run was classified by reading its diff against the landed fix and the tail of its transcript (Table 8).

***Table 8:*** *Failure classes of the failed local specification runs*

| Class | Qwen3-Coder-30B | Qwen3.6-35B |
|---|---|---|
| Edited the wrong file (usually the visible call site) | 7 | 1 |
| Right file, change incomplete | 5 | 0 |
| No edit at all | 3 | 0 |
| Broke the build | 1 | 0 |
| Ran out of time before editing | 0 | 14 |
| Tool misuse (tool call printed as text, wrong parameters) | 0 | 2 |
| Total failed | 16 | 17 |

The two models fail in different ways. Qwen3-Coder-30B works at a reasonable pace but changes the wrong place: told which component owns the behaviour, it still patches the line where the symptom shows. Qwen3.6-35B is too slow for the deadlines: at about one turn per minute it read and searched for most of its time and rarely reached the edit. In two runs it printed its tool calls as plain text, which ends the run. Writing the rule "fix the owner, not the call site" into the worker's instructions did not change the first behaviour; the instruction was present in all of these runs.

## 8 Results: Document Retrieval

Project B's model needs are modest in size and strict about privacy. Its benchmark was its own: 40 search queries with known relevant documents for the embedding models, a golden set of 34 questions and a held-out set of 24 for grounded answers, and date extraction on the first 40 documents of the archive.

***Table 9:*** *Embedding models for search (40 queries)*

| Model | Input limit | Recall@10 | MRR | Throughput |
|---|---|---|---|---|
| multilingual-e5-base (incumbent) | 512 tokens | 0.825 | 0.703 | – |
| bge-m3 | 8k tokens | 0.975 | 0.872 | 17.3 chunks/s |
| Qwen3-Embedding-0.6B | 32k tokens | 0.950 | 0.873 | 31.9 chunks/s |

Recall@10 is the share of queries whose relevant document appears among the first ten results; MRR is the mean reciprocal rank of the first relevant result (Manning, Raghavan, and Schütze, 2008). Both candidates are far better than the incumbent and equal to each other within the resolution of 40 queries (one query is 0.025 of recall). Throughput decides, because rebuilding the index over a growing archive is the expensive operation: Qwen3-Embedding-0.6B embeds 1.8 times faster: the 113,515 chunks of the current index re-embed in about an hour, and the planned rebuild, which also adds long books split into parts, was estimated at about 3.9 hours instead of about 7. The new index is built beside the old one and selected by configuration, so the change can be reverted without re-embedding.

For generation, larger models did not help. On grounded answers, the 8-billion-parameter Qwen3 model scored 33 of 34 on the golden set and 24 of 24 held out; a 24-billion-parameter model scored 34 of 34 and 22 of 24. On date extraction, Qwen3-8B found 18 of 40 dates at 11.5 s per document, the 24-billion-parameter model 9 of 40 at 40.7 s, and a 32-billion-parameter vision-language model 2 of 20 at 88.7 s. Served through MLX, Qwen3-8B was about twice as fast as through Ollama (prefill 1,149 against 438 tokens per second, decode 24 against 13). During the measurement Ollama was kept anyway, because two coding servers of about 28 GB each and parallel builds competed for memory, and Ollama loads the model on demand and unloads it when idle. That reason lapses with the routing decision of Section 10: with coding workers in the cloud, keeping the MLX model resident costs about 5 GB of 128 GB

and halves the latency of every answer, so a resident MLX server becomes the better default for Project B's generative jobs. The change is one configuration setting.

## 9 Discussion

*The serving stack is part of the model.* A local model judged through an unpatched server would have been judged on 30-second turns. Half of the night's effort went into serving: caching, memory budgets, output caps, retries and tool-argument repair. These fixes are general; they will serve the next model as well.

*Localisation, not editing, is the gap.* Where a local model was in the right function, it usually made a reasonable change. Where the right function was one step away from the symptom, it did not get there. This is consistent with the per-task pattern: the tasks the local models passed are exactly those in which symptom and owner are close together. It suggests that the precise prompt (file and function named) would raise the local pass rate on small fixes, and that it would not help with multi-file work. This report leaves it as a hypothesis for future work: the runs that would test it are prepared but were not part of this measurement (Section 12).

*A wrong specification costs a full run.* The view-decimals specification named the wrong cause. The cloud worker followed it faithfully and failed after 16.5 minutes. The cheapest worker is only cheap when the specification is right, which puts the weight on the orchestrator's diagnosis.

*Escalation is expensive when it waits for the deadline.* A failing local run usually ran to its deadline (15–40 minutes) before anything could fall back to the cloud. Short local time limits (10–15 minutes) are therefore part of any routing policy that tries local first; the successful local runs finished in 9–15 minutes.

*The machine is a shared resource.* With three local workers, two model servers and parallel Xcode builds, swap reached 17 of 18 GB and the load average 20–50, and thermal level 2 held for hours. On a 32 GB machine the realistic limit is one model server, one build and one worker; for Project A's build-heavy work that means the cloud.

## 10 Routing Policy

The decision taken from these results is configured with `pai worker policy` and summarised in Table 10.

***Table 10:*** *Routing decision after the measurement*

| Work | Route | Reason |
|---|---|---|
| Coding in general (multi-file, diagnosis, merges, anything with a gate test) | Cloud | 6/7 vs 3/19; local misses cost a full deadline |
| One-function fixes whose file and function the specification names; mechanical edits; read-only look-ups | Local first, 10–15 min limit, then cloud | Only class where local passed; saves subscription tokens, not time |
| Document embedding, extraction, answers | Always local | Privacy; local quality sufficient |
| Machine on battery or offline | Policy switch | `cloud-only` or `local-only` by one command |

## 11 Limitations

This is one night on one machine. The samples are small: seven cloud specification runs, 19 and 17 local ones. All coding tasks come from one code base and one subsystem, and from the two days before the measurement. The local models ran at 4-bit quantisation; higher precision might help and would cost memory. For most of the night the machine was at thermal level 2, which slowed local runs more than cloud runs, since only the former compute on the machine. The cloud worker was repeated less often than the local ones. The grader was itself fixed during the night (void runs, turn counts), and the early runs predate the serving fixes. The precise-specification variant, which would test the localisation hypothesis directly, was prepared but not run before the machine was needed for other work. Finally, the retrieval evaluation is Project B's own, with 40 queries.

*Does Claude grade itself?* A fair objection is that Claude models play several roles here: Claude Opus 5.5 orchestrated the night, built the tasks and wrote the specification prompts, and Claude Sonnet 5.5 is one of the contestants. Three things limit what that can do to the result. First, no model scores a run: a patch passes or fails on the hidden tests that shipped with the human-accepted fix, written before any of these runs, and the report task is scored by matching seven fixed facts. Second, the operator prompts are the maintainer's own words, not Claude's, and they show the same gap as the specification prompts (73 % against 7 % and 0 %). Third, every run's transcript, diff and grade is kept and can be re-graded by anyone with the repository. What remains is a subtler bias: the specifications were written by a Claude model and may suit a Claude worker's way of reading. The precise variant, which names only a file and a function, and an independent re-run with specifications written by someone else would bound it.

## 12 Conclusion and Future Work

On a 128 GB Apple M5 Max, today's open-weight models of the 30-billion-parameter class are not yet a replacement for a cloud coding worker on real application work: 16 % against 86 % with the same specification, with failures dominated by changing the wrong place. They are fully adequate for document retrieval, where a 0.6-billion-parameter embedding model outperformed the incumbent and an 8-billion-parameter model matched larger ones. The most useful outputs of the night are therefore the instruments: a replay benchmark with hidden tests, a serving stack that makes local agents responsive, guards and proof checks that catch what local workers get wrong, and a routing policy that can be changed by one command.

Next steps, in order: run the eight prepared precise-specification runs to test whether naming the file and function lifts local pass rates on small fixes; measure GLM-4.7-Flash (Z.ai, 2026), the strongest pure-attention candidate that fits, with `pai worker bench`; give local workers a tool that lists the callers and the owning type of a function, aimed at the dominant failure class; and measure the 32 GB profile with a build running beside the model server.

## Acknowledgements

The measurement, the instrumentation and this report were produced by Claude Code agents (Claude Opus 5.5 as orchestrator, Claude Sonnet 5.5 and the local models as workers) under the direction of the author, who set the goals and constraints and decided the outcome.

## References


Anthropic. (2026a). *Claude Code Overview*. [Online]. Available from: https://docs.anthropic.com/en/docs/claude-code/overview (Accessed: October 15, 2026).
(Cit. on p. 5.).

Anthropic. (2026b). *Models Overview: Claude Opus 5.5, Claude Sonnet 5.5 and Claude Haiku 4.5*. [Online]. Available from: https://docs.anthropic.com/en/docs/about-claude/models/overview (Accessed: October 15, 2026).
(Cit. on pp. 5, 7.).

Apple Machine Learning Research. (2026). *Exploring LLMs with MLX and the Neural Accelerators in the M5 GPU*. [Online]. Available from: https://machinelearning.apple.com/research/exploring-llms-mlx-m5 (Accessed: October 15, 2026).
(Cit. on p. 9.).

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. arXiv:2402.03216. [Online]. Available from: https://arxiv.org/abs/2402.03216 (Accessed: October 15, 2026).
(Cit. on p. 7.).

Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and Flexible Machine Learning on Apple Silicon*. [Online]. Available from: https://github.com/ml-explore/mlx (Accessed: October 15, 2026).
(Cit. on p. 6.).

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). 'SWE-bench: Can Language Models Resolve Real-World GitHub Issues?', *The Twelfth International Conference on Learning Representations*,
(cit. on p. 9.).

Kane, A. (2026). *pgvector: Open-Source Vector Similarity Search for Postgres*. [Online]. Available from: https://github.com/pgvector/pgvector (Accessed: October 15, 2026).
(Cit. on p. 6.).

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). 'Efficient Memory Management for Large Language Model Serving with PagedAttention', *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.
(Cit. on p. 6.).

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
(Cit. on p. 14.).

Mistral AI. (2025). *Devstral 2*. [Online]. Available from: https://docs.mistral.ai/models/devstral-2-25-12 (Accessed: October 15, 2026).
(Cit. on p. 7.).

Nott, M. (2026a). *AIBroker: A Message Bridge Between Claude Code Sessions and Messaging Channels*. [Online]. Available from: https://github.com/mnott/AIBroker (Accessed:

October 15, 2026).
(Cit. on p. 6.).

Nott, M. (2026b). *PAI: Personal AI Infrastructure for Claude Code*. [Online]. Available from: https://github.com/mnott/PAI (Accessed: October 15, 2026).
(Cit. on pp. 5, 6, 10.).

Ollama. (2026). *Ollama: Run Large Language Models Locally*. [Online]. Available from: https://ollama.com (Accessed: October 15, 2026).
(Cit. on p. 6.).

OpenAI. (2025). *gpt-oss-120b & gpt-oss-20b Model Card*. arXiv:2508.10925. [Online]. Available from: https://arxiv.org/abs/2508.10925 (Accessed: October 15, 2026).
(Cit. on p. 7.).

Qwen Team. (2025a). *Qwen3-Coder-30B-A3B-Instruct*. [Online]. Available from: https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct; 4-bit MLX conversion used: https://huggingface.co/lmstudio-community/Qwen3-Coder-30B-A3B-Instruct-MLX-4bit (Accessed: October 15, 2026).
(Cit. on p. 7.).

Qwen Team. (2025b). *Qwen3-Coder-Next*. [Online]. 4-bit MLX conversion used: https://huggingface.co/lmstudio-community/Qwen3-Coder-Next-MLX-4bit (Accessed: October 15, 2026).
(Cit. on p. 7.).

Qwen Team. (2026). *Qwen3.6-35B-A3B*. [Online]. 4-bit MLX conversion used: https://huggingface.co/mlx-community/Qwen3.6-35B-A3B-4bit (Accessed: October 15, 2026).
(Cit. on p. 7.).

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). 'Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer', *International Conference on Learning Representations*,
(cit. on p. 6.).

vllm-mlx contributors. (2026). *vllm-mlx: OpenAI- and Anthropic-Compatible Serving of MLX Models with Continuous Batching and Prefix Caching*. [Online]. Available from: https://github.com/waybarrios/vllm-mlx (Accessed: October 15, 2026).
(Cit. on p. 6.).

Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., & Wei, F. (2024). *Multilingual E5 Text Embeddings: A Technical Report*. arXiv:2402.05672. [Online]. Available from: https://arxiv.org/abs/2402.05672 (Accessed: October 15, 2026).
(Cit. on p. 7.).

Wilson, E. B. (1927). 'Probable Inference, the Law of Succession, and Statistical Inference', *Journal of the American Statistical Association*, 22 (158), 209–212.
(Cit. on pp. 2, 12.).

Yang, A., et al. (2025). *Qwen3 Technical Report*. arXiv:2505.09388. [Online]. Available from: https://arxiv.org/abs/2505.09388 (Accessed: October 15, 2026).
(Cit. on p. 7.).

Yang, S., Kautz, J., & Hatamizadeh, A. (2025). 'Gated Delta Networks: Improving Mamba2 with Delta Rule', *The Thirteenth International Conference on Learning Representations*,
(cit. on p. 6.).

Z.ai. (2026). *GLM-4.7-Flash*. [Online]. Available from: https://huggingface.co/zai-org/GLM-4.7-Flash (Accessed: October 15, 2026).
(Cit. on p. 17.).

Zhang, Y., et al. (2025). *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. arXiv:2506.05176. [Online]. Available from: https://arxiv.org/abs/2506.05176 (Accessed: October 15, 2026).
(Cit. on p. 7.).

Zheng, L., et al. (2024). 'SGLang: Efficient Execution of Structured Language Model Programs', *Advances in Neural Information Processing Systems*, 37.
(Cit. on p. 6.).